R65 Help Booklet

Contents

animate

```
ANIMATE

func expaint:boolean;
  Handles toggle graphics.
  Provides code to paint and apply motion.
  Returns true if animation loop should be
  stopped.,Stops if motion detects stop
  (e.g. expaint:=false)

func exkey(ch:char):boolean;
  Provides code to handle keys pressed.
  Returns true if animation loop should be
  stopped because of key pressed.
  (e.g. exkey:=ch=ord(0);)

Usage:
{$I IANIMATE:P}

animate(autorepeat)
  Run the animation loop.
  Set autorepeat to true if cursor key repetion
  should start immediately, not after short
  delay.
```

arglib

```
LIBRARY ARGLIB;

MEMORY
    NUMARG   = $005f: integer&;
    ARGLIST  = $0060: array[31] of integer;
    ARGLISTS = $0060: array[63] of char&;
    ARGTYPE  = $00a0: array[31] of char&;

VARIABLES
      _carg: integer;

ROUTINES
proc _argerror(e: integer);
proc _agetval(var value:integer; var default:boolean);
proc _agetstring(var string: array[15] of char;
  var default: boolean; var cyclus, drive: integer);
func option(opt:char):boolean;
```

bounce

```
BOUNCE - A Bouncing Ball Simulation

In this simulation, a ball bounces off the floor,
ceiling, walls, and a circle. Gravity and bounce
friction affect the motion of the ball.

The following modes are available:

  1. No circle
  2. Ball outside a small circle
  3. Ball inside a large circle

The simulation starts with a random velocity.

The following keys control the simulation:

  Up/Down     Increase/decrease vertical speed
  Right/Left  Increase/decrease horizontal speed
  Space       Select next mode
  Escape      Quit simulation
```

cat

```
CAT filename [firstline]

  display a text file, maximal 40 lines
  firstline: skip lines until first line
```

checkbuild

CHECKBUILD - Check All Pascal Programs

This command checks whether all Pascal programs can be compiled without errors. It is especially useful after modifying a library.

Use as follows:

```
  CD PSOURCE
  CHECKBUILD
  CD
```

A complete report is written to printout.txt.

The report is also displayed on the screen, but it will usually be larger than the terminal scroll buffer.

clean

```
CLEAN drive

   drive is the disk dive to clean, default 1

   - Deletes all except the latest version (cyclus)
     of each file
   - Deletes remaining compiler temporaries

   CLEAN does not free up any space. The deleted
   files remain on the disk. Use PACK to free up
   space after CLEAN.
```

compile

```
COMPILE name[.cy[,drv]] [/option]
  option:  P: no hard copy print
           L: put line info in object code
           R: range checking for index
           N: no loader file
           D: compile debug() commands
  Default drive is 1.

Compiler directives:
  {$I filename}   include file
  {$R-) {$R+}     range checking, default false
  {$U-} {$U+}     unique identifiers in scope
```

delete

```
DELETE name[.cy[,drv]] [/S]
name: name of Pascal source file
can include a subtype :X
supports wildcards
default drive is 1
default for subtype is :P
/S silent (no output on screen)

Delete one or several files
```

dir

```
DIR [drive] [name] [/s]

  where  /S        Sort directory
         /F        Full directory with dates
         /D        Include deleted files
         drive     Drive number, default 1
         name      name of a disk

  Examples  DIR              DIR /S
            DIR 0            DIR 0 /S
            DIR PSOURCE      DIR PSOURCE /S
```

edit

```
EDIT name[.cy[,drv]]
  name:   name of Pascal source file
          can include a subtype :X

  default drive is 1
  default for subtype is :P

  Edit a sequential file with the external editor.
```

errors

```
System Error Codes
==================
01      Read/write error
02      Checksum error
03      Escape exit during read/write
04      Record number error
05      File type error
06      File not found
07      Disk not ready
08      Directory full, file not stored
09      Illegal IRQ
10      Expression missing
11      Memory cell cannot be changed
12      Break table full, not inserted
13      Illegal memory cell for break
14      Double breakpoint setting
15      End of line expected
16      Syntax wrong in register name or =
17      Breakpoint not found in table
18      Syntax wrong in store
19      File subtype wrong or missing
20      Wrong file type, not runnable
21      Unknown monitor command
22      Illegal opcode for STEP/TRACE
23      Too many open files, not opened
24      Direction error in sequential read/write
25      Wrong file number, file not open
26      Disk full, file not stored
27      Random access index out of range
28      Illegal drive for random access
29      File not open for random access write

EXDOS Error Codes
=================
61      Wildcard not allowed
62      Only for disk, not tape
63      Illegal copy
64      File too large
65      Write error
66      Import error
67      Unknown emulator command
68      Unable to run external editor
69      File not changed or no new file
```

```
Pascal Runtime Error Codes
==========================
81      Division by zero
82      Stack overflow
83      Index out of bounds
84      Program not found or not a Pascal program
85      Illegal P-code
86      Escape during execution
87      No loader file made by compiler
88      Heap overflow
89      Pointer not allocated (NIL)
90      Writing to constant string not allowed
91      String too long (more than 63 characters)
92      Only the last allocated string can be released

Pascal Argument Error Codes
===========================
101     Argument is not string or default
102     Argument is not number or default
103     Argument is not starting with /
104     Unknown argument
105     Drive is not 0 or 1
106     Argument syntax error
107     Too many arguments
```

export

```
EXPORT name[.cy[,drv]]
  name:   name of Pascal source file
          can include a subtype :X

  default drive is 1
  default for subtype is :P

  Export a sequential file.
```

filelib

```
LIBRARY FILELIB;

MEMORY
    FILFLG = $00da: integer&;
    FILDRV = $00dc: integer&;
    FILTYP = $0300:char&;
    FILNAM = $0301: array[15] of char&;
    FILCYC = $0311: integer&;
    FILSTP = $0312: char&;
    FILNM1 = $0320: array[15] of char&;
    FILCY1 = $0330: integer&;

ROUTINES
func _uppercase(ch1: char): char;
proc _asetfile(name: array[15] of char;
      cyclus,device: integer; subtype: char);
proc _runprog(name0: array[15] of char;
             cyc: integer; drv: integer);
proc _chainprog(name0: array[15] of char;
             cyc: integer; drv: integer);
proc _write_label;
func _bestmatch: boolean;
func _freedrv(drive:integer; printit:boolean);
```

graph

```
GRAPH filename
  display the graph of function values
  stored in table on internal display

TEKGRAPH filename
  display the graph of function values
  stored in table on Tektronix 4010

MAKETABLE filename
  make a table for GRAPH/TEKGRAPH
```

graphics

```
LOW RESOLUTION GRAPHICS ON THE R65 COMPUTER

Graphics can be displayed in split mode in a
graphic overlay, or in fullscreen mode.

To toggle between full screen and split mode
use the key "ctrl-l".

To clear the graphics canvas and release the
graphics memory use the command "GREND".
```

grep

```
GREP "pattern" filename

Find pattern in text files.
If quotes are used, pattern can contain lower case
letters and other characters. Pattern without quotes
can only use upper case letters and numbers.
Filename can include wildcards , cyclus and drive.
Default is drive 1.
```

help

```
HELP topic

  where topic is a command name or another
  file with extention :H on drive 1 or 0.
```

ianimate

```
ROUTINES
proc animate(arepeat:boolean);
```

ichkesc

```
ROUTINES
func chkesc(checktoggle:boolean):boolean;
```

icircle

```
ROUTINES
proc circle(x,y,r,c:integer);
```

ifile

```
ROUTINES
proc runprog
  (name: array[15] of char;
   cyc: integer; drv: integer);
proc _writename(text: array[15] of char);
proc setsubtype(subtype:char);
func contains(t:array[7] of char):boolean;
func letter(ch:char):boolean;
proc setargs(name:array[15] of char;
  _carg0,cyc,drv:integer);
proc setargi(val,_carg0:integer);
```

import

```
IMPORT name[.cy[,drv]]
  name:   name of Pascal source file
          can include a subtype :X

  default drive is 1
  default for subtype is :P

  Import a sequential file
```

iotest

```
PROGRAM IOTEST
USES SYSLIB, STRLIB
VAR S1: CPNT
    FOUT: FILE
BEGIN
  WRITELN ('IOTEST')
  STRFIO('IOTEST:B',1)
  OPENW(FOUT)
  WRITELN(@FOUT, 'IOTEST')
  CLOSE(FOUT)
END.
```

irandom

```
ROUTINES
func rrandom(min,max:real):real;
func irandom(min,max:integer):integer;
```

keys

```
IMPORTANT KEYBOARD KEYS ON THE R65 SYSTEM

CTRL-l          split/full screen graphics
CTRL-r          print all on
CTRL-t          print all off

CTRL-<return>   clear to end of screen
CTRL-<right>    insert char
CTRL-<left>     delete char
CTRL-<down>     scroll down
CTRL-<up>       scroll up

SHIFT-CTRL      is required on UTM/Debian
                and on Macintosh
```

ladders

```
LADDERS  - Climb ladders and avoid a bouncing ball

You are entering a building with several floors at the
bottom left corner. There are ladders to climb up and
down between floors. There are also holes, over which
you can jump. The exit is at top right corner.
You win the game if you reach the exit, but there is
also a bouncing ball coming from the exit. The ball
can fall through and jump over the holes. You loose
if the ball hits you.

Command line parameter:
    /D endless demo mode

Keys
  left/right  move to the left/right
  up/down     climb ladders up/down
  ESC         quit the game

You jump automatically over holes if you are fast
enough.
```

ledlib

```
LIBRARY LEDLIB;

MEMORY
      LEDREG=$1432: array[7] of char&;

ROUTINES
proc _ledstring(s:cpnt);
proc _ledstop;
proc _ledhex(d,p,digits: integer);
proc _ledbyte(d: integer);
```

mathlib

```
LIBRARY MATHLIB;

CONSTANTS
      PI = 3.14159;
      E  = 2.71828;

ROUTINES
func fabs(x:real):real;
func sqrt(n:real):real;
func cos(x:real):real;
func sin(x:real):real;
func tan(x:real):real;
proc writeflo(r:real; fl:integer);
proc writef0(r: real; fl, d:integer;
    centered:boolean);
proc __wrfix(r:real; fl, d:integer);
func readflo(f:file):real;
func ln(r:real):real;
func exp(x:real):real;
func log(x:real):real;
```

nroff

```
NROFF(1)


NAME
====
NROFF - text formatter for R65

SYNOPSIS
========
NROFF filnam[:sub[.drv]] [linewidth]
Default line width is 40.

DESCRIPTION
===========
NROFF formats plain text files using simple dot
commands. Lines starting with a dot are
formatting requests.All other lines are normal
text and are formatted using fill mode.

TEXT MODES
==========
Fill modeWords are wrapped to fit the current
line width.
No fill modeText is printed exactly as entered.
.fi     Enable fill mode.
.nf     Enable no fill mode.

HEADINGS
========
.TH     Manual header line.
.SH     Section heading. Uses underline or
inverse video depending on width.
.SS     Subsection heading.

PARAGRAPHS
==========
.PP     Start new paragraph.
.br     Line break.
.sp [n] Insert blank lines.

INDENTATION
===========
```

```
.RS     Increase indentation.
.RE     Restore indentation.
.IP labelIndented paragraph. Label followed by
text with hanging indent.

TABS
====
.ta n1 n2 ...Set tab stops.
Tabs are expanded during output.
If no tabs are defined, default spacing is 8
cols.

TEXT STYLE
==========
.B text Bold line. On 40 columns inverse video
is used.

INPUT RULES
===========
Maximum input line length depends on buffer
size.
Control characters except TAB are ignored.

OUTPUT WIDTH
============
Line width may be given as optional parameter.
Example NROFF MANUAL:B,1 80
Widths above 40 use printer style headings.

TABLES
======
Tables are created using no fill mode and tabs.
Example:
COL1    COL2    COL3
====    ====    ====
AA      BBB     CCCC

LIMITATIONS
===========
No macros or registers.
No escape sequences.
Formatting is single pass.

AUTHOR
======
```

Written for the R65 system.
Implementation by R. Richarz and ChatGPT.

END
===
End of NROFF manual.

paint

```
PAINT image - paint in graphics canvas

  Paint with the following keys:
    right arrow   _move cursor right
    left arror    _move cursor left
    up arrow      _move cursor up
    down arrow     _move cursor down
    C             clear image
    p             paint a dot at cursor position

    L             drawing line mode
    R             drawing rectange mode
    S             drawing character mode
    return        _draw object and exit mode
    esc           exit mode without drawing

    W             write image to disk
    Q             write image to disk and quit
    K             kill program without writing to disk
```

pcodes

```
PCODES filname[disk,from,to]

  Display P-codes of a Pascal Program.
  Pascal source code and object code must be
  on the same drive.

  If the program has been compiled with the /L
  option (line numbers), pcode displays the
  source line together with the pcodes for this
  line, and pcodes of a specified range of
  lines (from/to) can be displayed. Use SETB
  and RESETB to set breakpoints.
```

pedit

```
PEDIT filename

 ESC              Enter command
 t          Go to top of file
 b          Go to bottom of file
 l n        Go to line n
 f          Find
 a          Find again
 z          Clear marks
 c n        Mark lines of copy
 p          Paste copied lines
 m          Move copied lines
 d n        Delete n lines
 w          Write file
 q          Write file and quit
 k          Kill (quit without writing)
 ?,h        Help
```

plotlib

```
LIBRARY PLOTLIB;

CONSTANTS
      XSIZE=223;
      YSIZE=117;
      XWORDS=28;
      WHITE=0;
      INVERSE=1;
      BLACK=2;
      PLOTDEV=@128;

MEMORY
      KEYPRESSED=$1785: char&;

VARIABLES
      _xcursor, _ycursor: integer;

ROUTINES
proc _delay10msec(time:integer);
func _syncscreen;
proc _grinit;
proc _grend;
proc _cleargr;
proc _fullview;
proc _splitview;
proc _plot(x,y,c:integer);
proc _move(x,y:integer);
proc _draw(x,y,c:integer);
proc _plotmap(x,y,m:integer);
proc _waitforKEY;
proc _plotif(x,y,mode: integer);
proc _circle(cx,cy,r,mode: integer);
proc _rectangle(x, y, sx, sy, mode: integer);
```

R65

```
R65 QUICK REFERENCE
===================

MONITOR COMMANDS
================
DIR     Directory listing.
COPY    Copy file between drives.
DELETE  Delete a file.
EDIT    Edit text file.
LOAD    Load file to memory.
STORE   Store memory sektion to disk.
GO      Execute machine code in memory.
DIS     Disassemble memory.
REG     Show CPU registers.
DATE    Show or set date.
FLOPPY  Select disk image.
IMPORT  Import host file.
EXPORT  Export file to host.
RUN     Run a maschine code program.

PASCAL
======
COMPILE Compile Pascal program.
EDIT    Edit a text file.
name    Execute Pascal program name.
LIB     Load Pascal library.

NROFF
=====
NROFF   Format text document.
.TH     Manual header.
.SH     Section heading.
.SS     Subsection heading.
.PP     New paragraph.
.IP     Indented paragraph.
.nf / .fiNo fill / fill mode.
.ta     Set tab stops.

SCREEN CONTROL
==============
CT-E    Clear line.
CT-RTN  Clear to end of display.
```

```
CT-RGT  Insert character.
CT-LFT  Delete character.
CTRL-A  Cursor home.
CT-UP   Roll up.
CT-DN   Roll down.
```

ralib

```
LIBRARY RALIB;

CONSTANTS
      FREAD   = $00;
      FWRITE  = $20;
      FNEW    = $30;
      FSILENT = $40;

ROUTINES
func _uppercase(ch:char):char;
func _attach(fname:array[15] of char;
  cyclus,drive,operation,size,start:integer;
  subtype:char):file;
func _getsize:integer;
proc _getword(device:file; address:integer;
  var word:integer);
proc _putword(device:file; address:integer;
  word:integer);
proc _getreal(device:file; address:integer;
  var rvalue:array[1] of %integer);
proc _putreal(device:file; address:integer;
  rvalue:array[1] of %integer);
```

restore

```
RESTORE filename

  Restore deleted files(s).
  Wildcars allowed.

  Use DIR /D to display deleted files
```

setb

```
SETB n - Set a breakpoint at line n

A program compiled with the /L option will break
at line n. Then the following keys are available

S    step
C    continue
R    reset breakpoint and continue
1-9  trace n lines
ESC  stop

RESETB clears the breakpoint.

PCODES command - Display P-codes of a command
```

snap

```
SNAP image - save a snapshot in image

Additional commands for images:
SHOW image - show image
PAINT image - paint image
```

snippets

```
{ check for options }
if ARGTYPE[_carg]='s' then begin
  sortit  := option('S');
  full    := option('F');
  if not (sortit or full) then begin
    writeln('option must be /S or /F or /FS');
    exit;
  end;
end;

{ find file entry with best match }
  writeln;
  if not _bestmatch then begin
    writeln(CUP,INVVID,'File not found',NORVID);
    exit;
  end;
  writeln;
```

spritelib

```
LIBRARY SPRITELIB;

CONSTANTS
      NSPRITES      = 64;
      FRAMES        = 3;
      S_CLEAR       = 0;
      S_STANDING    = 1;
      S_RIGHT       = 9;
      S_LEFT        = 32;
      S_UP          = 14;
      S_DOWN        = 35;
      S_JUMPR       = 38;
      S_JUMPL       = 41;
      S_TURN        = 19;
      S_CIRCLE      = 45;
      S_BALL        = 46;
      S_SMILEY      = 47;
      S_PIECE_WHITE = 51;
      S_PIECE_BLACK = 52;
      S_GEMS        = 53;

VARIABLES
      sprites: array[256] of integer;

ROUTINES
proc _readsprites(drv: integer);
proc _showsprite(x, y, index: integer);
```

starship

STARSHIP – A Starship Game

You command a starship while enemy ships attempt to destroy you. Your ship has several systems: scanner, shield, phaser, and warp engine. Each system consumes energy when used and can also be damaged.

If a system is damaged, its maximum available energy is reduced by the amount of damage. Energy is restored slowly over time, but one selected system can be recharged more quickly. Damaged systems can also be repaired.

The scanner detects enemy ships within its range, which depends on the scanner's energy level and repair status. Enemy positions are displayed on a radar-like screen, showing both angle and distance.

The phaser has a much shorter range, which also depends on the available energy and the repair status of the phaser. It is fired using the angle obtained from the scanner.

The shield becomes less effective when damaged and must be lowered before firing the phaser.

The condition of the warp engine determines the maximum distance the ship can warp. Both angle and distance must be specified.

The game ends when all enemy ships have been destroyed or when your own starship is destroyed.

striolib

```
LIBRARY STRIOLIB;

CONSTANTS
    NAMESIZE = 15;

ROUTINES
proc _checkfilerr;
proc _sgetstring(string: cpnt; var scarg: integer;
                  var default: boolean);
proc _ssetsubtype(sname: cpnt; subtype:char;
                  force: boolean);
proc _strfio(name:cpnt; cyclus, drive: integer);
proc _getdiskname(s: cpnt; drive: integer);
proc _change_disk(s: cpnt; drv: integer);
proc _s_runprog(name: cpnt; cyc, drv: integer);
proc _s_chainprog(name: cpnt; cyc, drv: integer);
```

strlib

```
LIBRARY STRLIB;

CONSTANTS
      STRSIZE=64;
      ENDMARK=chr(0);

ROUTINES
proc _runerr(e:integer);
func _new: cpnt;
proc _release(s: cpnt);
func _strlen(strin:cpnt):integer;
proc _strcpy(strin, strout:cpnt);
proc _stradd(strin,strinout:cpnt);
func _strcmp(s1,s2:cpnt):integer;
func _strpos(ch:char; s1:cpnt;
          start:integer): integer;
func _strread(f:file; s:cpnt;
          var ateof0:boolean):integer;
proc _hexstr(d:integer; s:cpnt);
proc _strdelc(pos:integer; s:cpnt);
proc _strinsc(ch:char;pos:integer;s:cpnt);
proc _intstr(n:integer;s:cpnt;fsize:integer);
```

syslib

```
LIBRARY SYSLIB;

CONSTANTS
  TAB8     = chr(9);
  HOM      = chr(1);
  CSC      = chr($11);
  LF       = chr($a);
  FF       = chr($c);
  CR       = chr($d);
  EOF      = chr($7f);
  CUP      = chr($1a);
  CLRLIN   = chr($17);
  NORVID   = chr($0b);
  INVVID   = chr($0e);
  PRTON    = chr($12);
  PRTOFF   = chr($14);
  TOPMEM   = $c780;
  MAXINT   = $7fff;
  INPUT    = @0;
  OUTPUT   = @0;
  KEY      = @1;
  PRINTER  = @1;
  ECEXPORT = 1;
  ECIMPORT = 2;
  ECEDIT   = 3;
  EC_CLR_PRTOUT = 9;

MEMORY
  RUNERR   = $000c: integer&;
  FILERR   = $00db: integer&;
  ENDSTK   = $000e: integer;
  IOCHECK  = $0023: boolean&;
  CURPOS   = $00ee: integer&;

VARIABLES
  _day:    integer;
  _month:  integer;
  _year:   integer;

ROUTINES
func _emulator(i:integer): integer;
func _getbcd(address: integer): integer;
proc _getdate;
```

```
proc _prtdate(device: file);
func _abs(x: integer): integer;
func _mod(x,n: integer): integer;
proc _tab(pos: integer);
proc _abort;
proc _prtext8(device: file; text: array[7] of char);
proc _prtext16(device: file; text: array[15] of char);
func _random: integer;
proc __wrintf(value, width: integer);
func _readint(f:file; var n:integer): boolean;
func _isesc:boolean;
```

table

```
graph
  display the graph of function values
  stored in table
table
  make a table for display with graph
```

teklib

```
LIBRARY TEKLIB;

CONSTANTS
      MAXX = 1023;
      MAXY = 780;
      MAXCOLUMNS = 74;
      MAXLINES   = 35;
      SOLID      = 1;
      DOTTED     = 2;
      DOTDASH    = 3;
      SHORTDASH  = 4;
      LONGDASH   = 5;
      PLOTTER    = @1;
      T_HALF     = 0;
      T_FULLV    = 1;
      T_GAMING   = 2;

VARIABLES
      _xs,_ys: integer;

ROUTINES
proc _delay10msec(time:integer);
proc _clearscreen;
proc _starttek(mode: integer);
proc _endtek;
proc _startdraw(x1,y1:integer);
proc _draw(x2,y2:integer);
proc _enddraw;
proc _drawvector(x1,y1,x2,y2:integer);
proc _drawrectangle(x1,y1,x2,y2:integer);
proc _moveto(x1,y1: integer);
proc _setchsize(size:integer);
proc _setlinemode(type:integer);
```

timelib

```
LIBRARY TIMELIB;

VARIABLES
      tenmillis,seconds,minutes,hours,
    difftenmillis: integer;

ROUTINES
proc gettime;
proc prttime(device: file);
func timediff: real;
```

view

```
VIEW filename

 ESC, q         Quit
 CUP, CDOWN     Scroll up/down
 CLEFT, CRIGHT  Scroll 12 lines up/down
 /, f           find string
 n              next hit
 ?, h           help
```

wildlib

```
LIBRARY WILDLIB;

CONSTANTS
      NAMESIZE   = 15;
      NUMENTRIES = 255;

ROUTINES
proc _findentry(var nm:array[NAMESIZE] of char;
       drv:integer;var ent: integer;
       var fnd,lst:boolean);
proc _sfindentry(name: cpnt; drv:integer;
       var ent: integer;
       var fnd,lst: boolean);
proc _writename(nm1:array[15] of char);
```

writelib

```
LIBRARY WRITELIB;

ROUTINES
func hexchar(i0: integer): char;
func hexb(i: integer): cpnt;
func hexw(i: integer): cpnt;
```